



Genie Code

Context-aware AI assistant

What is Genie Code?

- **Genie Code** (formerly Databricks Assistant) is an autonomous AI partner purpose-built for data work in Databricks.
- Deeply integrated with Unity Catalog.
 - Understand your complete data landscape, including tables, columns, and lineage.
- Designed for data teams to use every day, from experimentation and model development to production pipelines and BI dashboards.
- Genie Code can independently plan, execute, and maintain complex data workflows like pipelines and machine learning models

Capabilities

- **Code Generation**
 - Creates SQL, Python, and PySpark code from natural language prompts, even converting between languages (e.g., Pandas to PySpark).
- **Code Explanation & Fixing**
 - Explains complex code and identifies/fixes errors with suggested code (Quick Fix).
- **Context-Awareness**
 - Leverages Unity Catalog metadata (tables, schemas, comments, lineage) and notebook context for relevant suggestions.
- **Productivity Modes**
 - Offers Chat, Edit (inline suggestions), and Agent modes (automating multi-step tasks).
- **Data Discovery**
 - Helps find and understand data assets within your workspace.
- **Integrated Experience**
 - Available across notebooks, SQL editor, file editor with options to dock or move the pane.

Core Mechanisms

- **Multi-Model Routing**

- Instead of relying on a single LLM, it acts as an agentic router. It automatically selects the most appropriate model (whether a frontier LLM, open-source model, or a custom Databricks-hosted model) for each specific sub-task in your request.

- **Deep Unity Catalog Integration**

- It is grounded in Unity Catalog metadata. This allows it to "understand" your organization's specific data landscape—including table relationships, column descriptions, and lineage—to generate highly accurate queries and find relevant assets without manual input.

- **Agent Mode (Planning & Execution)**

- When given a high-level goal (e.g., "Build a dashboard to identify flight delay risks"), Genie Code creates a multi-step execution plan. It can create and edit multiple files, run code in serverless sandboxes, and inspect results to decide the next step.

- **Model Context Protocol (MCP)**

- It uses the open-standard Model Context Protocol to securely connect with external tools like GitHub, Jira, and Confluence, allowing it to fetch external context or update project tickets autonomously.

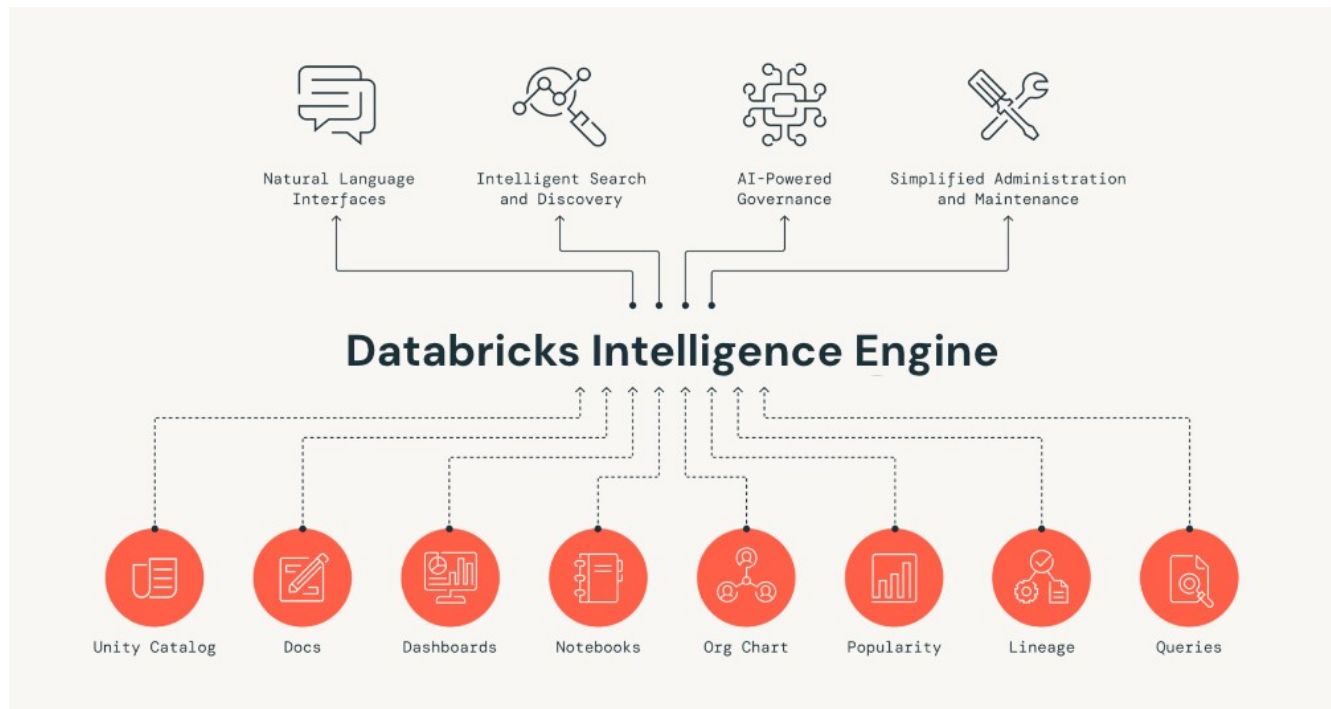
Core Architectural Components

- **Databricks Intelligence Engine**
 - This is the "brain" that provides the Assistant with knowledge of popular tables, schemas, and usage patterns across the entire company.
- **Unity Catalog Integration**
 - The Assistant uses Unity Catalog metadata—including column descriptions, tags, and comments—to ground its responses in the specific data assets of your organization.
- **Contextual Signals**
 - It continuously monitors real-time signals from the user's workspace, such as current notebook code cells, libraries, variables, and previous conversation history.

Data Intelligence Engine

Data Intelligence Engine (a.k.a **DatabricksIQ**) powers the entire Databricks Data Intelligence Platform.

Its the core AI-driven engine that learns from your organization's unique data, usage patterns, and business context to automate and simplify tasks.



Security and Governance

- Databricks Data Intelligence Engine is fully integrated with Unity Catalog, ensuring that AI-driven insights follow the organization's existing security and governance rules.
- Users only receive answers and see data that they are explicitly authorized to access.

Slash Commands

Databricks Assistant uses slash commands to quickly perform common development tasks. You can trigger these by typing / in the Assistant chat pane or using the inline assistant (Cmd+I / Ctrl+I).

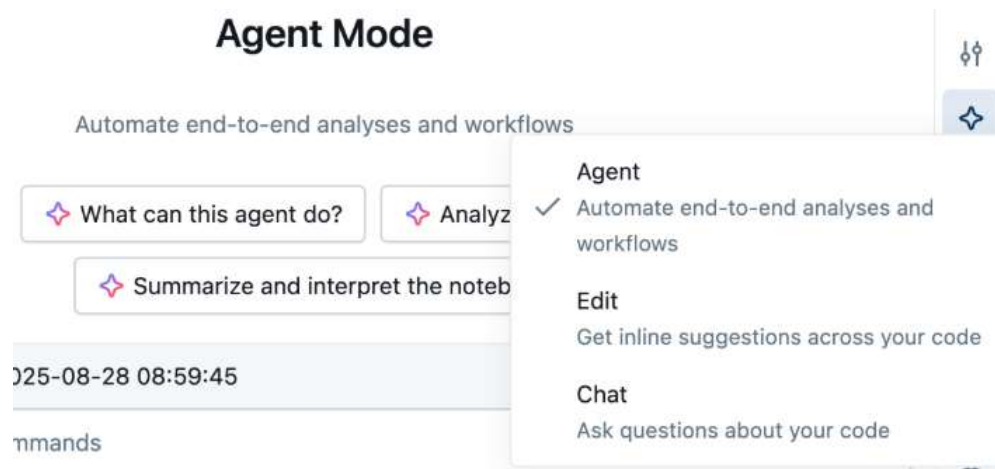
- **/**: Displays the menu of all available common commands.
- **/fix**: Analyzes the code in the current cell for errors and proposes a corrected version.
- **/explain**: Provides a natural language explanation of the logic within a code cell.
- **/optimize**: Evaluates and refactors SQL or Python code to improve performance.
- **/doc**: Automatically adds comments and documentation (docstrings) to the code.
- **/prettify**: Formats the selected code for better readability and style compliance.
- **/findTables**: Searches for relevant tables using Unity Catalog metadata.

<https://docs.databricks.com/aws/en/notebooks/code-assistant#use-slash-commands-for-prompts>

Interaction modes

Databricks Assistant offers four primary interaction modes designed for different stages of the data development lifecycle. These are accessible via a mode selector at the bottom of the Assistant pane or through specific keyboard shortcuts.

- **Chat Mode (Conversational)**
- **Agent Mode (Autonomous Execution)**
- **Inline Assistant (Cell-level Edits)**



Chat mode

The default interface for general assistance and rapid prototyping.

- **Best for:** Asking conceptual questions, getting code snippets, or searching documentation.
- **Key Feature:** Provides responses with citations from Databricks documentation and generates code (Python/SQL) that you can copy or insert directly into your notebook.
- **Context:** Uses the current notebook cell or SQL editor tab as context to personalize answers.

Agent mode

A (beta) feature known as the Data Science Agent that transforms the Assistant into an active collaborator.

- **Best for:** Automating multi-step workflows like full exploratory data analysis (EDA), training ML models, or building entire data pipelines.
- **Key Feature:** Unlike other modes, Agent Mode can plan steps, run code on serverless compute, analyze cell outputs, and automatically fix errors it encounters during execution. It asks for your confirmation before running code for security.

Inline Assistant

Triggered directly within a notebook code cell using Cmd+I (Mac) or Ctrl+I (Windows).

- **Best for:** Quick, focused edits within a specific block of code without opening the side panel.
- **Key Feature:** Opens a small text box inside the cell where you can type instructions like "add a bar chart for this data" or "fix the syntax error here".

Custom instructions

- **User Instructions**

- Open the Genie Code pane in the upper-right corner of your workspace.
- Click the Settings (gear) icon.
- Under "User instructions," click Add instructions file. This creates a **.assistant_instructions.md** file in your user directory.
- Add your instructions in plain English or Markdown.

- **Workspace Instructions:**

- Admins can create/edit a file named **.assistant_workspace_instructions.md** in the /Workspace directory.

Thank You